

SpaceVim

A community-driven Vim distribution

[Home](#) | [About](#) | [Documentation](#) | [Development](#) | [Community](#) | [Sponsors](#)

SpaceVim Documentation

» Features

- » [Modular configuration](#)
- » [Neovim centric - Dark powered mode](#)
- » [Language specific mode](#)
- » [Awesome ui](#)
- » [Unite centric work-flow](#)
- » [multiple leader mode](#)

» Custom configuration

- » [Layers](#)
- » [APIs](#)

Features

Awesome ui

» outline + filemanager + checker

```
12 import com.wsdjeg.chat.server.bot.SmartBot;
13 /**
14  * User public API:
15  * 1. '/join' : join or create a group
16  */
17 public class User {
18     private String userName;
19     private SmartBot smartBot;
20     private List<String> unreadMsg = new ArrayList<>();
21     private ServerThread client;
22     private List<User> friends = new ArrayList<>();
23     private List<Integer> groupIds = new ArrayList<>();
24
25     public User(String userName){
26         this.userName = userName;
27     }
28
29     public void join(String gName){
30         int id = GroupManager.getId(gName);
31         if (id != 0) {
32             if (!groupIds.contains(id)) {
33                 groupIds.add(id);
34             }
35         }
36     }
37
38     public void clearUnreadMsg() {
39         unreadMsg.clear();
40     }
41
42     public void getFriends() {
43         List<User> friends = new ArrayList<>();
44         for (Integer groupId : groupIds) {
45             List<User> groupMembers = GroupManager.getMembers(groupId);
46             for (User user : groupMembers) {
47                 if (!friends.contains(user)) {
48                     friends.add(user);
49                 }
50             }
51         }
52     }
53
54     public void getUnreadMsg() {
55         List<String> unreadMsg = new ArrayList<>();
56         for (Integer groupId : groupIds) {
57             List<String> groupUnreadMsg = GroupManager.getUnreadMsg(groupId);
58             for (String msg : groupUnreadMsg) {
59                 if (!unreadMsg.contains(msg)) {
60                     unreadMsg.add(msg);
61                 }
62             }
63         }
64     }
65
66     public void hasSameGroup(User user) {
67         for (Integer groupId : groupIds) {
68             List<User> groupMembers = GroupManager.getMembers(groupId);
69             for (User groupMember : groupMembers) {
70                 if (groupMember.equals(user)) {
71                     return true;
72                 }
73             }
74         }
75         return false;
76     }
77
78     public void isFriend(User user) {
79         for (User friend : friends) {
80             if (friend.equals(user)) {
81                 return true;
82             }
83         }
84         return false;
85     }
86
87     public void join(String gName) {
88         join(gName);
89     }
90
91     public void removeFriend(User name) {
92         for (User friend : friends) {
93             if (friend.userName.equals(name)) {
94                 friends.remove(friend);
95             }
96         }
97     }
98
99     public void send(String msg) {
100         smartBot.send(msg);
101     }
102
103     public void send(User sender, String msg) {
104         smartBot.send(sender.userName, msg);
105     }
106
107     public void setClient(ServerThread client) {
108         this.client = client;
109     }
110
111     public void setSmartBot(SmartBot smartBot) {
112         this.smartBot = smartBot;
113     }
114
115     public void setUserName(String userName) {
116         this.userName = userName;
117     }
118 }
```

Language specific mode

Key Mapping



1. code completion: autocompletion and fuzzy match.

```

1 #! /usr/bin/perl -w
2
3 functions
4 check_and_set_isatty($param) {
5     check_swap_exists($param)
6     win_init_size();
7     // Set the 'diff' option now, so that it can be checked for in a vimrc
8     // file. There is no buffer yet though.
9     if ($param.diff == 0) {
10         diff_win_options(firstwin, FALSE);
11         diw|
12     }
13     diff_win_options
14     a _PTRDIFF_WIDTH_ T_MAX;
15     exe_commands($param.T *
16     build_str hl int
17     close_ding_bef curs win ers +/
18     changed_line_abv curs win
19     T changed_window_setting void
20     changed_window_setting void
21     check_arg_idx void
22     n check_need_swap void
23     clearFolding void
24     close_windows highlight groups +/
25     foldFoldingState void
26     did_swapwrite.msg int
27     did_wait_return int ed., +/
28     debug_break_level = $param.use_debug_break_level;
29
30 bool reading_input = !$params.headless && ($params.input_isatty
31 || ! $params.output_isatty || $params.err_isatty);
32
33 if (reading_input) {
34     // One of the startup commands (arguments, sourced scripts or plugins) may
35     // be bz2baas <main.c> nvim_main() C c utf-8[unix] 14% 283/1888 9 vimfiler B

```

2. syntax check: Asynchronous linting and make framework.

[illegible]

go support

1. code completion:

```
17 }
18 func (slice importPackages) Len() int {
19     return len(slice)
20 }
21 func (slice importPackages) Less(i, j int) bool {
22     return slice[i].path < slice[j].path
23 }
24 func (slice importPackages) Swap(i, j int) {
25     slice[i], slice[j] = slice[j], slice[i]
26 }
27 const travis_yaml = ".travis.yml"
28
29 func genTravisVal() error {
30     _, err := os.Stat(
31         if err == nil { Stat(name string) (os.FileInfo, error) func
32         return e PathListSeparator bool func
33         IsPathSeparator(c uint8) bool func
34         f, err := os.Lstat(name string) (os.FileInfo, error) func
35         if err != nil { StartProcess(name string, argv []string, attr *os.ProcAttr) (*os.Process, error) func
36         return e ProcessState struct type
37     }
38     defer f.Close()
39     state := WriteString
40     go: statecraft's
41     - tip: statecraft's
42     before_install: stated
43     - go get github: statehood
44     tagbar Name gen.go INSERT master statehood's
45 -- INSERT --
```

2. syntax check:

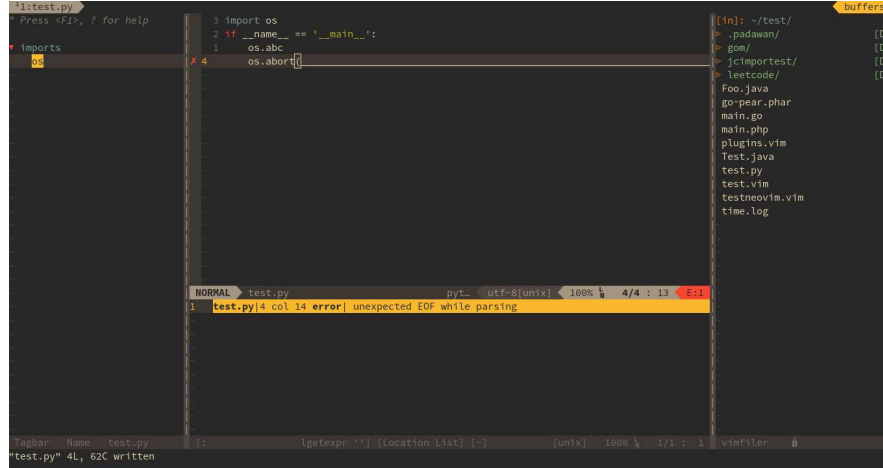
```
12 import (
13     "io/ioutil"
14     "os"
15     "path/filepath"
16     "runtime"
17     "strconv"
18     "strings"
19     "testing"
20 )
21 func TestExec(t *testing.T) {
22     dir, err := ioutil.TempDir("", "gom")
23     if err != nil {
24         t.Fatal(err)
25     }
26     defer os.RemoveAll(dir)
27     cwd, err := os.Getwd()
28     if err != nil {
29         t.Fatal(err)
30     }
31     // master exec_test.go TestExec() < go utf-8(unix) 17% 14/78 : 4
32     main.go[10 col 2 error] cannot find package "github.com/hashicorp/go-version" in any of:
33     1 | | ./ /usr/lib/go/src/github.com/hashicorp/go-version (from $GOROOT)
34     2 | | ./ /home/wsdjeg/goprojects/src/github.com/hashicorp/go-version (from $GOPATH)
35     3 | main.go[11 col 2 error] cannot find package "github.com/matt/gover" in any of:
36     4 | | ./ /usr/lib/go/src/github.com/matt/gover (from $GOROOT)
37     5 | | ./ /home/wsdjeg/goprojects/src/github.com/matt/gover (from $GOPATH)
38     6 | gomfile.go[175 warning] unreachable code
39 tagbar Name exec_test.go [setLocList()] [Location List] [-] [unix] 14% 14/78 : 1 vimfiler 8
```

python support

1. code completion:

```
2 import os
3 if __name__ == '__main__':
4     os.get_blocking(fd) function [jedi]
5     get_exec_path(env=None) function [jedi]
6     get_inheritable() function [jedi]
7     get_terminal_size(columns, lines) function [jedi]
8     getcwd() function [jedi]
9     getcwdb() function [jedi]
10    getegid() function [jedi]
11    getenv(key, default=None) function [jedi]
12    getenvb(key, default=None) function [jedi]
13    geteuid() function [jedi]
14    getgid() function [jedi]
15    getgrouplist(user, group) function [jedi]
16    getgroups() function [jedi]
17    getloadavg() function [jedi]
18    getlogin() function [jedi]
19 INSERT lang/python test.py[+] python [unix] 100% 3/3 : 10
```

2. syntax check:



The screenshot shows the Neovim editor interface in dark mode. The main window displays a Python file named 'test.py' with the following code:

```
1 import os
2 if __name__ == '__main__':
3     os.abc
4     os.abort()
```

A red error message is visible at the bottom of the editor: `1 [test.py]4 col 14 error] unexpected EOF while parsing`. The status bar at the bottom indicates the file is named 'test.py', is 4 lines long, and 62 characters have been written. The right sidebar shows a list of buffers, including `test.py`.

Neovim centric - Dark powered mode of SpaceVim.

By default, SpaceVim use these dark powered plugins:

1. [deoplete.nvim](#) - Dark powered asynchronous completion framework for neovim
2. [dein.vim](#) - Dark powered Vim/Neovim plugin manager

TODO:

1. [defx.nvim](#) - Dark powered file explorer
2. [deoppet.nvim](#) - Dark powered snippet plugin
3. [denite.nvim](#) - Dark powered asynchronous unite all interfaces for Neovim/Vim8

Modular configuration

Multiple leader mode

Global origin vim leader, default : \

Vim's origin global leader can be used in all modes.

Local origin vim leader, default : ,

Vim's origin local leader can be used in all the mode.

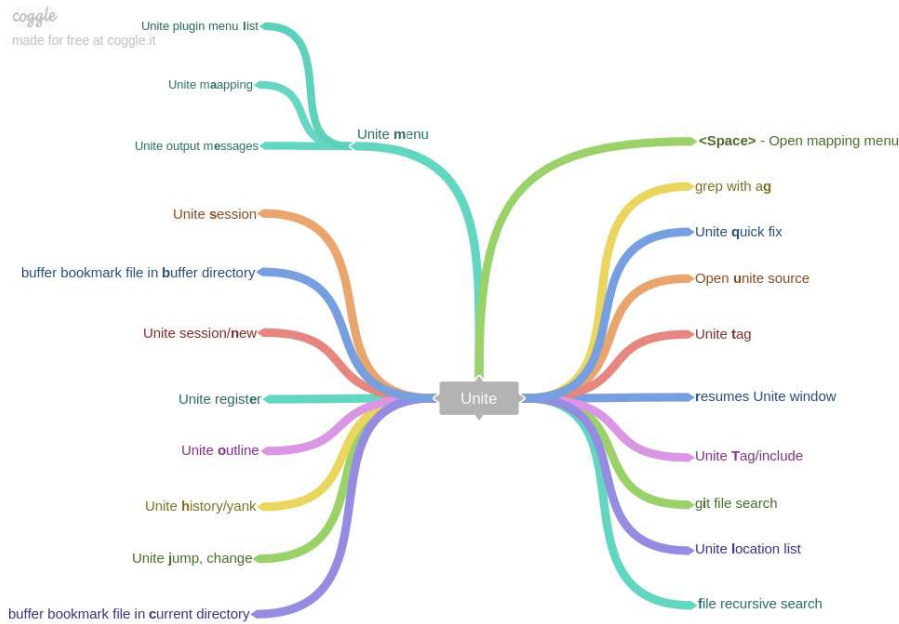
Windows function leader, default : s

Windows function leader can only be used in normal mode. For the list of mappings see the [link](#)

Unite work flow leader, default : f

Unite work flow leader can only be used in normal mode. Unite leader need unite groups.

Unite centric work-flow



» List all the plugins has been installed, fuzzy find what you want, default action is open the github website of current plugin. default key is <leader>lp

```

1: README.md
7 [[Facebook]](https://img.shields.io/badge/Facebook-SpaceVim-blue.svg) (https://www.facebook.com/SpaceVim)
6
5 [[GitHub_watchers]](https://img.shields.io/github/watchers/SpaceVim/SpaceVim.svg?style=social&label=Watch) (https://github.com/SpaceVim/SpaceVim)
4 [[GitHub_stars]](https://img.shields.io/github/stars/SpaceVim/SpaceVim.svg?style=social&label=Star) (https://github.com/SpaceVim/SpaceVim)
3 [[GitHub_forks]](https://img.shields.io/github/forks/SpaceVim/SpaceVim.svg?style=social&label=Fork) (https://github.com/SpaceVim/SpaceVim)
2 [[Twitter_Follow]](https://img.shields.io/twitter/follow/SpaceVim.svg?style=social&label=Follow&maxAge=2592000) (https://twitter.com/SpaceVim)
1
10 [[2017-01-07_136x723]](https://cloud.githubusercontent.com/assets/13142418/21739011/6c38c14e-d4ca-11e6-8f3e-2c21dfc489c0.png)
1
2 [SpaceVim](https://github.com/SpaceVim/SpaceVim) is a Modular configuration, a bundle of custom settings and plugins for Vim,
3 here we call them layers, each layer has different plugins and config, user just need
4 to select the layers they need. It got inspired by [spacemacs](https://github.com/syl20bnr/spacemacs). If you use SpaceVim,
5 please star it on github. It's a great way of getting feedback and gives me the kick to
README.md
[autocomplete] [Shougo/neosnippet-snippets] [lazy loaded] [{"merged": 0}]
[autocomplete] [Shougo/neoinclude.vim] [lazy loaded] [{"on_f": 1}]
[autocomplete] [Shougo/context_filetype.vim] [lazy loaded] [{"on_f": 1}]
[autocomplete] [Shougo/neco-vim] [lazy loaded] [{"on_f": 1, "loadconf_before": 1}]
[autocomplete] [Shougo/neco-syntax] [lazy loaded] [{"on_f": 1}]
[denite] [Shougo/denite.nvim] [lazy loaded] [{"loadconf": 1, "merged": 0}]
[unite] [Shougo/vimfiler.vim] [lazy loaded] [{"loadconf": 1, "loadconf_before": 1, "merged": 0}]
[unite] [Shougo/unite-session]
[unite] [Shougo/unite-help]
[unite] [Shougo/neobundle-vim-recipes]
[unite] [Shougo/unite-outline]
[unite] [Shougo/neomru.vim]
[unite] [Shougo/neoyank.vim]
[unite] [Shougo/unite.vim] [lazy loaded] [{"loadconf": 1, "merged": 0}]
[core] [Shougo/vimproc.vim] [lazy loaded] [{"build": "make"}]
[Lang] [Shougo/neosnippet.vim] [lazy loaded] [{"on_f": 1, "loadconf": 1, "on_ft": "neosnippet"}]
2 shougo
[unite] default menu: AddedPlugins(18/208) | All the Added plugins <leader>lp [unite] - default
-- INSERT --
  
```

» List all the mappings and description: f<space>

```

1: README.md > f:SpaceVim.txt f:SpaceVim.vim
2 " SpaceVim use ~/.SpaceVim.d/init.vim as it's default global config file.
3 " you can get all the SpaceVim options and layers in it. ~/.SpaceVim.d/ will
4 " also be added to runtimepath, so you can write your own script in it.
5 " SpaceVim also support local config of each project. It should be
6 " ~/.SpaceVim.d/init.vim in the root directory of your project, and
7 " ~/.SpaceVim.d/ will also be added to runtimepath.
18
1 ""
2 " Version of SpaceVim, this value can not be changed.
3 scriptencoding utf-8
4 let g:spacevim_version = '0.1.0-dev'
5 let <S>spacevim_version = g:spacevim_version
6 ""
autoload/SpaceVim.vim
> Close current windows [Window]q
> Switch to the last buffer [Window]l
> Empty current buffer [Window]x
> Close other windows [Window]o
> Create new tab [Window]t
> vsplit window [Window]g
> split window [Window]v
> vsplit vertically, switch to next window [Window]p
> Use s as window prefix [Window]s
> Define window prefix [Window]
> Open previous buffer in vsplit window <Leader>sg
> Open previous buffer in split window <Leader>sv
> Smart close windows q
> Switch to previous window or tab <S-Tab>
> Switch to next window or tab <Tab>
[unite]
tagbar Name SpaceVim.vim <| Custom mapped keyboard shortcuts [unite]<SPACE> [unite] - default vimfiler ß
-- INSERT --
  
```

» List all the starred repos in github.com, fuzzy find and open the website of the repo. default key is <leader>ls

```
1: README.md - 2: SpaceTime.vim
6
6
6
4 - List all the mappings and description: <F6>space>
3
3
2
1 - List all the starred repos in github.com, fuzzy find and open the website of the repo. default key is '<leader>ls'
214
214
2
2
3
4 #### Plugin Highlights
5
6 - Package management with caching enabled and lazy loading
markdown utf-8(unix) 3,238 words < 30% 214/711 : 1
gopherjs/gopherjs A compiler from Go to JavaScript for running Go code in a browser
AlDaniel/cloc cloc counts blank lines, comment lines, and physical lines of source code in many programming languages.
xoreos/xoreos A reimplementation of BioWare's Aurora engine (and derivatives). Pre-pre-alpha :P
matze/vim-move Plugin to move lines and selections up and down
ruanyf/coverage.vim code coverage vim plugin
majutsushi/tagbar Vim plugin that displays tags in a window, ordered by scope
vincentdoovie Documentation generator for Vim plug-ins
lamddai/sue/gina.vim Alpha: Git manipulation plugin for Neovim/Vim8 (new version of vim-gita)
sbdchd/neoformat :sparkles: A (Neo)Vim plugin for formatting code.
zchee/nvim-go Go development plugin for Neovim written in pure Go
WolfgangMehner/vim-plugins Vim plug-ins which offer support for various programming languages.
ternjs/tern_for_vim Tern plugin for Vim
Rip-Rip/clang_complete Vim plugin that use clang for completing C/C++ code.
css/perlowm1.vim perl omnicompletion for vim (including base class function completions .. etc)
tweekmonster/exception.vim Vim plugin for tracing exceptions thrown by VimL scripts.
mattboehm/vim-unstack Vim plugin for parsing stack traces and opening the files
plugin|
unite default menu:MyStarredrepos(49/344) | All github repos starred by me <leader>ls [unite] - default
-- INSERT --
```

Plugin Highlights

- » Package management with caching enabled and lazy loading
- » Project-aware tabs and label
- » Vimfiler as file-manager + SSH connections
- » Go completion via vim-go and gocode
- » Javascript completion via Tern
- » PHP completion, indent, folds, syntax
- » Python jedi completion, pep8 convention
- » Languages: Ansible, css3, csv, json, less, markdown, mustache
- » Helpers: Undo tree, bookmarks, git, tmux navigation, hex editor, sessions, and much more.

Note that 90% of the plugins are **[lazy-loaded]**. [lazy-loaded]: ./config/plugins.vim

Non Lazy-Loaded Plugins

Name	Description
dein.vim	Dark powered Vim/Neovim plugin manager
vimproc	Interactive command execution
colorschemes	Awesome color-schemes
file-line	Allow opening a file in a given line
neomru	MRU source for Unite
cursorword	Underlines word under cursor
gitbranch	Lightweight git branch detection
gitgutter	Shows git diffs in the gutter
tinyline	Tiny great looking statusline
tagabana	Central location for all tags
bookmarks	Bookmarks, works independently from vim marks
tmux-navigator	Seamless navigation between tmux panes and vim splits

Lazy-Loaded Plugins

Language

Name	Description
html5	HTML5 omnicomplete and syntax
mustache	Mustache and handlebars syntax
markdown	Markdown syntax highlighting
ansible-yaml	Additional support for Ansible
jinja	Jinja support in vim

Name	Description
less	Syntax for LESS
css3-syntax	CSS3 syntax support to vim's built-in <code>syntax/css.vim</code>
csv	Handling column separated data
pep8-indent	Nicer Python indentation
logstash	Highlights logstash configuration files
tmux	vim plugin for tmux.conf
json	Better JSON support
toml	Syntax for TOML
i3	i3 window manager config syntax
Dockerfile	syntax and snippets for Dockerfile
go	Go development
jedi-vim	Python jedi autocompletion library
ruby	Ruby configuration files
portfile	Macports portfile configuration files
javascript	Enhanced Javascript syntax
javascript-indent	Javascript indent script
tern	Provides Tern-based JavaScript editing support
php	Up-to-date PHP syntax file
phpfold	PHP folding
phpcomplete	Improved PHP omnicompletion
phpindent	PHP official indenting
phpspec	PhpSpec integration

Commands

Name	Description
vimfiler	Powerful file explorer
[NERD Commenter]	Comment tool - no comment necessary
vinarise	Hex editor
syntastic	Syntax checking hacks
gita	An awesome git handling plugin
gista	Manipulate gists in Vim
undotree	Ultimate undo history visualizer
incsearch	Improved incremental searching
expand-region	Visually select increasingly larger regions of text
open-browser	Open URI with your favorite browser
prettyprint	Pretty-print vim variables
quickrun	Run commands quickly
ref	Integrated reference viewer
dictionary	Dictionary.app interface
vimwiki	Personal Wiki for Vim
thesaurus	Look up words in an online thesaurus

Commands

Name	Description
goyo	Distraction-free writing
limelight	Hyperfocus-writing
matchit	Intelligent pair matching
indentline	Display vertical indention lines
choosewin	Choose window to use, like tmux's 'display-pane'

Completion

Name	Description
delimitmate	Insert mode auto-completion for quotes, parenthesis, brackets
echodoc	Print objects' documentation in echo area
deoplete	Neovim: Dark powered asynchronous completion framework
neocomplete	Next generation completion framework
neosnippet	Contains neocomplete snippets source

Unite

Name	Description
unite	Unite and create user interfaces
unite-colorscheme	Browse colorschemes
unite-filetype	Select file type
unite-history	Browse history of command/search
unite-build	Build with Unite interface
unite-outline	File "outline" source for unite
unite-tag	Tags source for Unite
unite-quickfix	Quickfix source for Unite
neossh	SSH interface for plugins
unite-pull-request	GitHub pull-request source for Unite
junkfile	Create temporary files for memo and testing
unite-issue	Issue manager for JIRA and GitHub

Operators & Text Objects

Name	Description
operator-user	Define your own operator easily
operator-replace	Operator to replace text with register content
operator-surround	Operator to enclose text objects
textobj-user	Create your own text objects
textobj-multiblock	Handle multiple brackets objects

Custom Key bindings

Key	Mode	Action
<leader> + y	visual	Copy selection to X11 clipboard ("+y)
Ctrl + c	Normal	Copy full path of current buffer to X11 clipboard

Key	Mode	Action
<leader> + Ctrl + c	Normal	Copy github.com url of current buffer to X11 clipboard(if it is a github repo)
<leader> + Ctrl + l	Normal/visual	Copy github.com url of current lines to X11 clipboard(if it is a github repo)
<leader> + p	Normal/visual	Paste selection from X11 clipboard ("+p)
Ctrl + f	Normal	Smart page forward (C-f/C-d)
Ctrl + b	Normal	Smart page backwards (C-b/C-u)
Ctrl + e	Normal	Smart scroll down (3C-e/j)
Ctrl + y	Normal	Smart scroll up (3C-y/k)
Ctrl + q	Normal	Ctrl + w
Ctrl + x	Normal	Switch buffer and placement
Up,Down	Normal	Smart up and down
}	Normal	After paragraph motion go to first non-blank char (j^)
<	Visual/Normal	Indent to left and re-select
>	Visual/Normal	Indent to right and re-select
Tab	Visual	Indent to right and re-select
Shift + Tab	Visual	Indent to left and re-select
gp	Normal	Select last paste
Q / gQ	Normal	Disable EX-mode ()
Ctrl + a	Command	Navigation in command line
Ctrl + b	Command	Move cursor backward in command line
Ctrl + f	Command	Move cursor forward in command line

File Operations

Key	Mode	Action
<leader> + cd	Normal	Switch to the directory of the open buffer
<leader> + w	Normal/visual	Write (:w)
Ctrl + s	Normal/visual/Command	Write (:w)
:w!!	Command	Write as root (!sudo tee > /dev/null %)

Editor UI

Key	Mode	Action
F2	All	Toggle tagbar
F3	All	Toggle Vimfiler
<leader> + num	Normal	Jump to the buffer whit the num index
<Alt> + num	Normal	Jump to the buffer whit the num index, this only works in neovim
<Alt> + h / <Left>	Normal	Jump to left buffer in the tabline, this only works in neovim
<Alt> + l / <Right>	Normal	Jump to Right buffer in the tabline, this only works in neovim
<leader> + ts	Normal	Toggle spell-checker (:setlocal spell!)

Key	Mode	Action
<leader> + tn	Normal	Toggle line numbers (:setlocal nonumber!)
<leader> + tl	Normal	Toggle hidden characters (:setlocal nolist!)
<leader> + th	Normal	Toggle highlighted search (:set hlsearch!)
<leader> + tw	Normal	Toggle wrap (:setlocal wrap! breakindent!)
g0	Normal	Go to first tab (:tabfirst)
g\$	Normal	Go to last tab (:tablast)
gr	Normal	Go to previous tab (:tabprevious)
Ctrl + <Down>	Normal	Move to split below (j)
Ctrl + <Up>	Normal	Move to upper split (k)
Ctrl + <Left>	Normal	Move to left split (h)
Ctrl + <Right>	Normal	Move to right split (l)
*	Visual	Search selection forwards
#	Visual	Search selection backwards
, + Space	Normal	Remove all spaces at EOL
Ctrl + r	Visual	Replace selection
<leader> + lj	Normal	Next on location list
<leader> + lk	Normal	Previous on location list
<leader> + s	Normal/visual	Source selection

Window Management

Key	Mode	Action
q	Normal	Smart buffer close
s + p	Normal	Split nicely
s + v	Normal	:split
s + g	Normal	:vsplit
s + t	Normal	Open new tab (:tabnew)
s + o	Normal	Close other windows (:only)
s + x	Normal	Remove buffer, leave blank window
s + q	Normal	Closes current buffer (:close)
s + Q	Normal	Removes current buffer, left buffer in the tabline will be displayed, if there is no buffer on the left, the right buffer will be displayed, if this is the last buffer in the tabline, then an empty buffer will be displayed.
Tab	Normal	Next window or tab
Shift + Tab	Normal	Previous window or tab
<leader> + sv	Normal	Split with previous buffer
<leader> + sg	Normal	Vertical split with previous buffer

SpaceVim has mapped normal q as smart buffer close, the normal func of q can be get by ``` q`

Native functions

Key	Mode	Action
<leader> + qr	Normal	Same as native q
<leader> + qr/	Normal	Same as native q/ , open cmdwin
<leader> + qr?	Normal	Same as native q? , open cmdwin
<leader> + qr:	Normal	Same as native q: , open cmdwin

Plugin: Unite

Key	Mode	Action
[unite]	Normal	unite leader, default is f , :hg:spacevim_unite_leader
[unite] + r	Normal	Resumes Unite window
[unite] + f	Normal	Opens Unite file recursive search
[unite] + i	Normal	Opens Unite git file search
[unite] + g	Normal	Opens Unite grep with ag (the_silver_searcher)
[unite] + u	Normal	Opens Unite source
[unite] + t	Normal	Opens Unite tag
[unite] + T	Normal	Opens Unite tag/include
[unite] + l	Normal	Opens Unite location list
[unite] + q	Normal	Opens Unite quick fix
[unite] + e	Normal	Opens Unite register
[unite] + j	Normal	Opens Unite jump, change
[unite] + h	Normal	Opens Unite history/yank
[unite] + s	Normal	Opens Unite session
[unite] + n	Normal	Opens Unite session/new
[unite] + o	Normal	Opens Unite outline
[unite] + c	Normal	Opens Unite buffer bookmark file in current directory
[unite] + b	Normal	Opens Unite buffer bookmark file in buffer directory
[unite] + ma	Normal	Opens Unite mapping
[unite] + <space>	Normal	Opens Unite menu:CustomKeyMaps
[unite] + me	Normal	Opens Unite output messages
<leader> + bl	Normal	Opens Unite buffers, mru, bookmark
<leader> + ta	Normal	Opens Unite tab
<leader> + ugf	Normal	Opens Unite file with word at cursor
<leader> + ugt	Normal/visual	Opens Unite tag with word at cursor
Within <i>Unite</i> buffers		
Ctrl + h/k/l/r	Normal	Un-map
Ctrl + r	Normal	Redraw
Ctrl + j	Insert	Select next line

Key	Mode	Action
Ctrl + k	Insert	Select previous line
,	Normal	Toggle mark current candidate, up
e	Normal	Run default action
Ctrl + v	Normal	Open in a split
Ctrl + s	Normal	Open in a vertical split
Ctrl + t	Normal	Open in a new tab
Tab	Normal	Ctrl + w + w
Escape	Normal	Exit unite
jj	Insert	Leave Insert mode
r	Normal	Replace ('search' profile) or rename
Tab	Insert	Unite autocompletion
Ctrl + z	Normal/insert	Toggle transpose window
Ctrl + w	Insert	Delete backward path

Plugin: VimFiler

Key	Mode	Action
<F3>	Normal	Toggle file explorer
Within <i>VimFiler</i> buffers		
Ctrl + j	Normal	Un-map
Ctrl + l	Normal	Un-map
E	Normal	Un-map
sv	Normal	Split edit
sg	Normal	Vertical split edit
p	Normal	Preview
i	Normal	Switch to directory history
v	Normal	Quick look
gx	Normal	Execute with vimfiler associated
,	Normal	Toggle mark current line
V	Normal	Clear all marks
Ctrl + r	Normal	Redraw

Plugin: neocomplete

Key	Mode	Action
Enter	Insert	Smart snippet expansion
Ctrl + space	Insert	Autocomplete with Unite
Tab	Insert/select	Smart tab movement or completion
Ctrl + j/k/f/b	Insert	Movement in popup
Ctrl + g	Insert	Undo completion
Ctrl + l	Insert	Complete common string
Ctrl + o	Insert	Expand snippet

Key	Mode	Action
Ctrl + y	Insert	Close pop-up
Ctrl + e	Insert	Close pop-up
Ctrl + l	Insert	Complete common string
Ctrl + d	Insert	Scroll down
Ctrl + u	Insert	Scroll up

Plugin: NERD Commenter

Key	Mode	Action
<leader> + cc	Normal/visual	Comment out the current line or text selected in visual mode.
<leader> + cn	Normal/visual	Same as cc but forces nesting.
<leader> + cu	Normal/visual	Uncomments the selected line(s).
<leader> + cs	Normal/visual	Comments out the selected lines with a pretty block formatted layout.
<leader> + cy	Normal/visual	Same as cc except that the commented line(s) are yanked first.

Plugin: Goyo and Limelight

Key	Mode	Action
<leader> + G	Normal	Toggle distraction-free writing

Plugin: ChooseWin

Key	Mode	Action
-	Normal	Choose a window to edit
<leader> + -	Normal	Switch editing window with selected

Plugin: Bookmarks

Key	Mode	Action
m + a	Normal	Show list of all bookmarks
m + m	Normal	Toggle bookmark in current line
m + n	Normal	Jump to next bookmark
m + p	Normal	Jump to previous bookmark
m + i	Normal	Annotate bookmark

As SpaceVim use above bookmarks mappings, so you can not use `a`, `m`, `n`, `p` or `i` registers to mark current position, but other registers should works will. if you really need to use these registers, you can add `nnoremap <leader>m m` to your custom configuration, then you use `use a` registers via `\ma`

Plugin: Gita

Key	Mode	Action
<leader> + gs	Normal	Git status
<leader> + gd	Normal	Git diff
<leader> + gc	Normal	Git commit
<leader> + gb	Normal	Git blame

Key	Mode	Action
<leader> + gp	Normal	Git push

Plugin: vim-signify

Key	Mode	Action
<leader> + hj /]c	Normal	Jump to next hunk
<leader> + hk / [c	Normal	Jump to previous hunk
<leader> + hJ /]C	Normal	Jump to last hunk
<leader> + hK / [C	Normal	Jump to first hunk

Misc Plugins

Key	Mode	Action
<leader> + gu	Normal	Open undo tree
<leader> + i	Normal	Toggle indentation lines
<leader> + j	Normal	Start smalls
<leader> + r	Normal	Quickrun
<leader> + ?	Normal	Dictionary
<leader> + w	Normal	Wiki
<leader> + K	Normal	Thesaurus

Custom configuration

SpaceVim use `~/.Spacevim.d/init.vim` as default global init file, you can set SpaceVim-options or config layers in it. SpaceVim also will add `~/.Spacevim.d/` into runtimepath, so you can write your own vim script in it.

SpaceVim also support local config file for project, the init file is `.Spacevim.d/init.vim` in the root of your project. `.Spacevim.d/` will also be added into runtimepath.

here is an example config file for SpaceVim:

```
" Here are some basic customizations, please refer to the ~/.Spacevim.d/init.vim
" file for all possible options:
let g:spacevim_default_indent = 3
let g:spacevim_max_column    = 80

" Change the default directory where all miscellaneous persistent files go.
" By default it is ~/.cache/vimfiles.
let g:spacevim_plugin_bundle_dir = '~/.cache/vimfiles'

" set SpaceVim colorscheme
let g:spacevim_colorscheme = 'jellybeans'

" Set plugin manager, you want to use, default is dein.vim
let g:spacevim_plugin_manager = 'dein' " neobundle or dein or vim-plug

" use space as `<Leader>`
let mapleader = "\<space>"

" Set windows shortcut leader [window], default is `s`
let g:spacevim_windows_leader = 's'

" Set unite work flow shortcut leader [Unite], default is `f`
let g:spacevim_unite_leader = 'f'
```

```
" By default, language specific plugins are not loaded. This can be changed
" with the following, then the plugins for go development will be loaded.
call SpaceVim#layers#load('lang#go')

" loaded ui layer
call SpaceVim#layers#load('ui')

" If there is a particular plugin you don't like, you can define this
" variable to disable them entirely:
let g:spacevim_disabled_plugins=[
    \ ['junegunn/fzf.vim'],
    \ ]

" If you want to add some custom plugins, use these options:
let g:spacevim_custom_plugins = [
    \ ['plasticboy/vim-markdown', {'on_ft' : 'markdown'}],
    \ ['wsdjeg/GitHub.vim'],
    \ ]

" set the guifont
let g:spacevim_guifont = 'DejaVu\ Sans\ Mono\ for\ Powerline\ 11'
```

Comprehensive documentation is available for each layer by `:h SpaceVim`.